

CENTRO UNIVERSITÁRIO DE BRASÍLIA – CEUB

Pós-Graduação em Engenharia de Inteligência Artificial

Disciplina: Engenharia de Aprendizado de Máquina

ExoHunter-ML

Detecção de Exoplanetas em Curvas de Luz
do Telescópio Kepler via Aprendizado Supervisionado

Projeto de Engenharia e Operacionalização de Machine Learning

Trilha A – Aprendizado Supervisionado (Classificação)

Autor(a):	Jaysa Gabrielly
Professor(a):	Jonathan de Moura Feitosa
Repositório:	https://github.com/el-pitchula/ExoHunter-ML
Link do Colab:	notebook
Modelagem matemática:	notebook

Brasília – DF
2026

Sumário

1	Introdução	2
2	Descrição do problema	2
3	Dataset utilizado	3
4	Metodologia adotada	4
5	Pipeline de Machine Learning	4
5.1	Ingestão dos dados	4
5.2	Análise exploratória (EDA)	5
5.3	Engenharia de atributos	5
5.4	Normalização, padronização e codificação	5
5.5	Separação treino / validação / teste	5
5.6	Treinamento e ajuste de hiperparâmetros	6
6	Resultados experimentais	7
6.1	Desempenho no conjunto de validação	7
6.2	Desempenho no conjunto de teste	7
7	Análise das métricas	8
8	Explicabilidade do modelo	9
9	Estratégia de monitoramento	9
10	Conclusão	10
	Referências	12

1 Introdução

Este relatório documenta o desenvolvimento completo do projeto **ExoHunter-ML**, uma solução de Machine Learning para detecção de exoplanetas a partir de curvas de luz estelares observadas pelo telescópio espacial *Kepler*, da NASA. O trabalho foi desenvolvido como projeto final da disciplina de Engenharia de Aprendizado de Máquina, seguindo a Trilha A – Aprendizado Supervisionado (Classificação).

O objetivo geral é aplicar, de ponta a ponta, o ciclo de vida de um modelo de Machine Learning em um cenário realista de engenharia: desde a compreensão do problema e a escolha de um conjunto de dados público, passando pela preparação dos dados, engenharia de atributos, treinamento e ajuste de hiperparâmetros, até a avaliação crítica dos resultados, a explicabilidade do modelo e a proposta de uma estratégia de monitoramento em produção.

O projeto foi desenvolvido em Python, no ambiente Google Colab, e está organizado em dois notebooks complementares:

- **Notebook entregável** (`ExoHunter_ML.ipynb`): apresenta o pipeline final, limpo e direto, do problema à proposta de monitoramento – é este notebook que acompanha este relatório;
- **Notebook de experimentos** (`ExoHunter_ML(lab).ipynb`): um registro do processo investigativo completo, incluindo tentativas descartadas, depurações e a análise aprofundada dos erros do modelo, usado como material de apoio para a apresentação.

2 Descrição do problema

Quando um planeta cruza, do nosso ponto de vista, a frente de sua estrela hospedeira – um evento chamado **trânsito** –, uma fração pequena e periódica da luz da estrela é bloqueada. Esse método de detecção, conhecido como **método de trânsito**, foi a técnica utilizada pela missão Kepler para identificar milhares de candidatos a exoplanetas ao longo de quase uma década de observação contínua de mais de 150 mil estrelas.

O problema abordado neste projeto consiste em: **dada uma curva de luz (série temporal de medições de fluxo estelar), classificar se essa estrela possui ou não evidência de um planeta em trânsito**. Trata-se, portanto, de um problema de **classificação binária supervisionada**, com duas características centrais que moldam toda a abordagem de engenharia adotada:

1. **Forte desbalanceamento de classes**: trânsitos são eventos raros – a grande maioria das estrelas observadas não apresenta nenhum planeta detectável em trânsito;

2. **Sinal sutil:** mesmo quando existe um planeta, a queda no brilho é da ordem de fração de 1% a poucos por cento do fluxo total, facilmente confundida com ruído instrumental.

Esses dois fatores tornam o problema um bom exercício de engenharia de ML: não basta treinar um classificador qualquer – é necessário lidar explicitamente com desbalanceamento, escolher métricas de avaliação adequadas (accuracy sozinha é enganosa) e construir uma representação de atributos que capture o padrão físico do fenômeno.

3 Dataset utilizado

Foi utilizado o conjunto de dados público **Kepler Labelled Time Series Data**, disponível no Kaggle:

<https://www.kaggle.com/datasets/keplersmachines/kepler-labelled-time-series-data>

O carregamento é feito de forma totalmente reprodutível via biblioteca **kagglehub**, sem necessidade de autenticação local e sem caminhos de arquivo fixos, atendendo ao requisito de reprodutibilidade do projeto.

Estrutura dos dados

- Cada linha representa uma estrela observada; cada uma das 3.197 colunas `FLUX.1` a `FLUX.3197` é uma medição de fluxo luminoso em um instante de tempo;
- A coluna `LABEL` indica a classe: 2 = estrela com exoplaneta confirmado, 1 = sem exoplaneta;
- Conjunto de treino: 5.087 curvas; conjunto de teste: 570 curvas (arquivos `exoTrain.csv` e `exoTest.csv`, respectivamente).

Qualidade dos dados

A verificação inicial não identificou nenhum valor ausente, infinito ou linha duplicada em nenhum dos dois conjuntos, conforme resumido na Tabela 1.

Tabela 1: Verificação de qualidade dos dados

Indicador	Valor
Amostras de treino	5.087
Amostras de teste	570
Pontos por curva	3.197
Valores ausentes (treino / teste)	0 / 0
Valores infinitos (treino / teste)	0 / 0
Duplicatas (treino / teste)	0 / 0

Desbalanceamento de classes

O dataset apresenta um desbalanceamento extremo entre as classes, conforme mostrado na Tabela 2: apenas 0,73% das estrelas do conjunto de treino possuem exoplaneta confirmado.

Tabela 2: Distribuição de classes

Classe	Treino		Teste	
	Quantidade	%	Quantidade	%
Sem exoplaneta	5.050	99,27%	565	99,12%
Com exoplaneta	37	0,73%	5	0,88%

Esse desbalanceamento é o fator determinante de várias decisões de engenharia descritas nas seções seguintes, em especial a escolha das métricas de avaliação e o uso de ponderação de classes no treinamento do modelo.

4 Metodologia adotada

O projeto segue a **Trilha A – Aprendizado Supervisionado (Classificação)**, utilizando o algoritmo **Random Forest**. A escolha se justifica por três motivos práticos:

1. lida bem com atributos numéricos de escalas diferentes, sem exigir padronização;
2. captura relações não lineares entre os atributos estatísticos extraídos das curvas;
3. fornece, nativamente, uma medida de importância de atributos, o que facilita a etapa de explicabilidade.

O desenvolvimento seguiu uma metodologia experimental iterativa: um notebook de experimentos foi utilizado para testar hipóteses, investigar o desbalanceamento, comparar abordagens de engenharia de atributos (com e sem normalização) e depurar problemas de implementação, antes de consolidar o pipeline final – limpo e sem os caminhos descartados – no notebook entregável, que é o objeto central deste relatório.

5 Pipeline de Machine Learning

5.1 Ingestão dos dados

Os dados são baixados via `kagglehub.dataset_download()` e carregados com `pandas.read_csv()`, sem qualquer caminho local fixo, garantindo que o notebook seja executável por terceiros sem modificação.

5.2 Análise exploratória (EDA)

Além da checagem de qualidade e da distribuição de classes (Seção 3), foi realizada uma inspeção visual de curvas de luz individuais de cada classe, permitindo observar a diferença de comportamento entre estrelas com e sem trânsito detectável.

5.3 Engenharia de atributos

Utilizar diretamente os 3.197 pontos de fluxo como entrada do modelo geraria um problema severo de alta dimensionalidade frente a apenas 37 exemplos positivos no treino, além de manter todo o ruído bruto do sinal. A solução adotada foi resumir cada curva em um vetor de **15 atributos estatísticos**:

- **Tendência central:** mean, median
- **Dispersão:** std, mad, range
- **Extremos:** min, max
- **Quantis:** q01, q05, q25, q75, q95, q99
- **Forma da distribuição:** skewness, kurtosis

Essa escolha não é arbitrária: como um trânsito afeta apenas uma fração pequena dos pontos de uma curva, ele desloca a mediana muito pouco, mas cria uma cauda assimétrica de valores extremos – exatamente o que os quantis inferiores, o **skewness** negativo e a **kurtosis** elevada capturam estatisticamente.

5.4 Normalização, padronização e codificação

Como o Random Forest é invariante a escala (não é sensível à magnitude absoluta dos atributos, apenas à sua ordenação relativa em cada divisão de árvore), **não foi necessário aplicar normalização ou padronização** aos atributos numéricos. A única codificação aplicada foi a da variável-alvo: o rótulo original (1 = sem exoplaneta, 2 = com exoplaneta) foi recodificado para o padrão binário 0/1.

5.5 Separação treino / validação / teste

A base de treino original (5.087 curvas) foi dividida de forma estratificada em treino (80%) e validação (20%), preservando a proporção de classes. O conjunto de teste (**exoTest.csv**, 570 curvas) foi mantido completamente isolado, utilizado apenas na avaliação final.

Tabela 3: Tamanho dos conjuntos após engenharia de atributos

Conjunto	Dimensão
Treino	4.069×15
Validação	1.018×15
Teste	570×15

5.6 Treinamento e ajuste de hiperparâmetros

O modelo foi treinado com `class_weight="balanced"`, para compensar o desbalanceamento diretamente no processo de divisão das árvores. Os hiperparâmetros não foram escolhidos manualmente: foi utilizada `RandomizedSearchCV` com validação cruzada de 3 folds, otimizando o **F1-score** (métrica que equilibra precisão e recall, mais adequada que accuracy dado o desbalanceamento).

Listing 1: Configuração da busca de hiperparâmetros

```

1 param_distributions = {
2     "n_estimators": [200, 300, 400],
3     "max_depth": [None, 10, 20, 30],
4     "min_samples_split": [2, 5, 10],
5     "min_samples_leaf": [1, 2, 4],
6     "max_features": ["sqrt", "log2", None]
7 }
8
9 search = RandomizedSearchCV(
10     rf, param_distributions=param_distributions,
11     n_iter=12, scoring="f1", cv=3,
12     random_state=SEED, n_jobs=-1
13 )

```

Os melhores hiperparâmetros encontrados, com F1 médio de **0,8403** na validação cruzada, foram:

Tabela 4: Melhores hiperparâmetros (`RandomizedSearchCV`)

Hiperparâmetro	Valor
<code>n_estimators</code>	400
<code>max_depth</code>	10
<code>min_samples_split</code>	2
<code>min_samples_leaf</code>	1
<code>max_features</code>	None

6 Resultados experimentais

6.1 Desempenho no conjunto de validação

Tabela 5: Métricas na validação

Métrica	Valor
Accuracy	1,0000
Precision	1,0000
Recall	1,0000
F1-score	1,0000
ROC-AUC	1,0000
Average Precision	1,0000

O desempenho perfeito na validação deve ser interpretado com cautela: o conjunto de validação contém apenas cerca de 7 exemplos positivos (20% dos 37 exemplos positivos do treino original), de modo que qualquer métrica de recall/precision nesse conjunto tem alta variância – um único exemplo classificado incorretamente já alteraria substancialmente os números. O resultado é, portanto, mais um indicativo de que o modelo captura bem o padrão nos dados de validação do que uma garantia de desempenho perfeito em produção. A avaliação no conjunto de teste, isolado e nunca utilizado no ajuste do modelo, é a referência mais confiável.

6.2 Desempenho no conjunto de teste

Tabela 6: Métricas no teste (conjunto isolado)

Métrica	Valor
Accuracy	0,9965
Precision	1,0000
Recall	0,6000
F1-score	0,7500
ROC-AUC	0,9547
Average Precision	0,6263

Tabela 7: Relatório de classificação detalhado – conjunto de teste

Classe	Precision	Recall	F1-score	Suporte
Sem exoplaneta	0,9965	1,0000	0,9982	565
Com exoplaneta	1,0000	0,6000	0,7500	5
Accuracy (geral)		0,9965		570
Macro avg	0,9982	0,8000	0,8741	570
Weighted avg	0,9965	0,9965	0,9961	570

Em termos absolutos: dos 5 exoplanetas reais presentes no conjunto de teste, o modelo identificou corretamente **3** (60%), sem gerar nenhum falso positivo (*precision* = 100%). Os gráficos de matriz de confusão correspondentes estão disponíveis no notebook anexado ao final deste documento.

7 Análise das métricas

A escolha das métricas de avaliação foi diretamente determinada pelo desbalanceamento de classes descrito na Seção 3. Uma accuracy de 99,65% no teste, isoladamente, seria enganosa: um classificador trivial que nunca prevê “exoplaneta” atingiria uma accuracy de aproximadamente 99,12% no mesmo conjunto, sem ter aprendido absolutamente nada. Por isso, o foco da avaliação recai sobre **Precision**, **Recall**, **F1-score** e **ROC-AUC**.

Precision perfeita (1,0): nenhum falso positivo foi gerado no teste – toda vez que o modelo sinaliza um candidato a exoplaneta, o sinal é real. Do ponto de vista de um pipeline de triagem científica, isso é valioso, pois reduz o esforço de validação manual por especialistas.

Recall de 0,60: o modelo identifica 3 dos 5 candidatos reais no teste, deixando 2 sem detecção (falsos negativos). Esse é o principal ponto de atenção do modelo – a investigação detalhada desses casos (disponível no notebook de experimentos) mostrou que os falsos negativos não são explicados de forma simples por “trânsitos rasos”; a análise de importância de atributos (SHAP) por instância aponta que, nesses casos específicos, a queda de fluxo, mesmo quando profunda, fica concentrada em poucos pontos e por isso não desloca suficientemente a mediana da curva – a estatística de maior peso no modelo.

ROC-AUC de 0,9547: como essa métrica avalia a capacidade do modelo de *ranquear* corretamente positivos acima de negativos, independentemente do limiar de decisão, um valor próximo de 1 indica que o modelo separa bem as classes em termos de probabilidade – mesmo quando a decisão final (com limiar padrão de 0,5) erra alguns casos.

Em suma, o modelo prioriza **não gerar falsos alarmes** em detrimento de capturar 100% dos casos positivos – um trade-off razoável para um cenário de triagem, mas que motiva a estratégia de monitoramento de recall descrita na Seção 9.

8 Explicabilidade do modelo

Foram utilizadas **duas técnicas complementares** de explicabilidade: a importância de atributos nativa do Random Forest (*Feature Importance*) e valores de **SHAP** (*SHapley Additive exPlanations*), calculados com `shap.TreeExplainer`.

Tabela 8: Comparação das técnicas de explicabilidade – 5 atributos mais relevantes

Atributo	Feature Importance	SHAP (média valor)
median	0,5278	0,3247
skewness	0,1120	0,0372
q01	0,0863	0,0341
mean	0,0642	0,0248
q99	0,0496	0,0130

As duas técnicas concordam de forma consistente: a **mediana** (`median`) domina disparadamente a decisão do modelo, seguida por `skewness` e `q01`. Essa concordância entre uma medida global (*Feature Importance*) e uma medida local agregada (SHAP) reforça a confiabilidade da interpretação.

Interpretação física do resultado: como um trânsito afeta apenas uma fração pequena dos pontos de uma curva, ele quase não desloca a mediana em relação a uma curva sem trânsito – exceto quando o evento é grande o suficiente para deixar uma marca estatística perceptível mesmo nessa medida robusta. O fato de a mediana ser o atributo mais importante sugere que o modelo aprendeu, essencialmente, a distinguir curvas cujo “nível de repouso” já se desloca de forma anômala – um proxy razoável, mas não perfeito, da presença de um trânsito, o que também explica a origem dos falsos negativos discutidos na Seção 7.

9 Estratégia de monitoramento

Para uma eventual implantação em produção, propõe-se o acompanhamento contínuo dos indicadores listados na Tabela 9, cobrindo tanto a saúde dos dados de entrada (*data drift*) quanto o comportamento do modelo (*prediction drift*) e seu desempenho preditivo.

Tabela 9: Plano de monitoramento em produção

Indicador	Objetivo	Ação em caso de alerta
Distribuição das features	Detectar <i>data drift</i>	Investigar mudança nos dados de entrada
Distribuição das probabilidades	Detectar <i>prediction drift</i>	Investigar alteração no comportamento do modelo
Recall	Monitorar detecção de positivos	Avaliar necessidade de retreinamento
Precision	Monitorar falsos alarmes	Investigar qualidade dos dados e do modelo
F1-score	Acompanhar desempenho geral	Comparar com o baseline reportado neste relatório
Taxa de falsos negativos	Controlar o principal risco do sistema	Investigar prioritariamente
Volume de novas observações	Verificar alterações no fluxo de aquisição	Verificar pipeline de coleta de dados

Estratégia de retreinamento

Recomenda-se um ciclo de retreinamento **disparado por evento** (e não puramente periódico), acionado quando: (i) o F1-score em uma amostra de validação contínua cair abaixo de um limiar definido em relação ao baseline reportado ($F1 = 0,75$ no teste); (ii) a distribuição das 15 features estatísticas apresentar desvio significativo (por exemplo, via teste de Kolmogorov–Smirnov) em relação à distribuição de treino; ou (iii) um volume suficiente de novos exemplos rotulados (confirmados por especialistas) se acumular, permitindo reforçar o aprendizado sobre a classe minoritária. Dado o baixo volume de exemplos positivos disponíveis, cada novo caso confirmado tem valor desproporcionalmente alto para o retreinamento do modelo.

10 Conclusão

Este projeto desenvolveu, de ponta a ponta, um pipeline de Machine Learning supervisionado para detecção de exoplanetas em curvas de luz do telescópio Kepler, cobrindo desde a ingestão reprodutível dos dados até a proposta de monitoramento em produção. Os principais resultados foram:

- Um modelo Random Forest, com hiperparâmetros otimizados via `RandomizedSearchCV`, capaz de identificar candidatos a exoplanetas com **precisão perfeita** (nenhum falso positivo) e **recall de 60%** no conjunto de teste isolado;
- Uma engenharia de atributos fundamentada fisicamente, reduzindo cada curva de 3.197 pontos a 15 estatísticas interpretáveis, sem prejuízo relevante de desempenho frente ao uso dos dados brutos;

- Duas técnicas de explicabilidade (Feature Importance e SHAP) convergindo para a mesma conclusão: a mediana da curva é o fator mais determinante da decisão do modelo, uma consequência direta de como um trânsito afeta estatisticamente a distribuição de fluxo;
- Uma proposta concreta de monitoramento e retreinamento, adequada às limitações de volume de dados da classe minoritária.

Como principal limitação, destaca-se que a abordagem de features estatísticas agregadas descarta a ordem temporal dos dados – não captura diretamente a periodicidade nem a forma exata do trânsito, o que provavelmente explica os falsos negativos observados. Como trabalho futuro, sugere-se explorar as Trilhas C (CNN 1D) ou D (RNN/LSTM) do enunciado, que poderiam extrair diretamente da série temporal bruta os padrões de periodicidade e formato do trânsito atualmente inacessíveis a este pipeline.

Referências

1. Kaggle. *Kepler Labelled Time Series Data*. Disponível em: <https://www.kaggle.com/datasets/keplersmachines/kepler-labelled-time-series-data>.
2. NASA. *Kepler Mission Overview*. Disponível em: <https://science.nasa.gov/mission/kepler/>.
3. Repositório do projeto: <https://github.com/el-pitchula/ExoHunter-ML>.